金融助手项目介绍与实现

这个系统是一个完整的RAG（检索增强生成）解决方案，专门针对财务报告等结构化文档进行智能问答。

1.系统架构特点

1.1模块化设计

- **文档处理模块**: 专门处理PDF解析
- **向量化模块**: 负责文本分块和嵌入
- **检索模块**: 实现相似度搜索
- **问答模块**: 整合检索和生成

1.2.数据处理流程

```
PDF文档 → MinerU解析 → JSON结构化数据 → 文本分块 → 向量化 → 向量存储

→用户问题 → 向量检索 → 上下文构建 → LLM生成 → 答案返回
```

1.3 关键技术

1. **MinerU**: 专业的文档布局分析工具
2. **ChromaDB**: 轻量级向量数据库
3. **DashScope**: 阿里云的AI服务，提供嵌入和生成模型
4. **LangChain**: 简化了嵌入模型的使用

1.4 优化策略

- **智能分块**: 考虑文档结构（标题、表格）
- **表格处理**: 特殊分块策略保持表格完整性
- **元数据丰富**: 存储额外的文档信息
- **错误处理**: API调用的异常处理

2.环境依赖

- **环境变量**:
  - `DASHSCOPE_API_KEY`: 阿里云API密钥
  - `EMBEDDING_MODEL`: 嵌入模型名称（默认text-embedding-v2）
- **主要依赖库**:
  - `mineru`: 文档解析
  - `chromadb`: 向量数据库
  - `langchain`: AI应用框架
  - `dashscope`: 阿里云AI服务SDK

3.代码详解

1. 导入模块

```
import glob  # 文件通配符匹配，目的匹配到目标文件
import os  # 操作系统接口
import json  # JSON数据处理
import subprocess  # 运行子进程
from datetime import datetime  # 日期时间处理
import chromadb  # 向量数据库
import torch  # PyTorch深度学习框架
import doclayout_yolo.nn.tasks  # MinerU的YOLO模型相关
import dashscope  # 阿里云灵积API
from http import HTTPStatus  # HTTP状态码

# 添加安全反序列化类，确保加载YOLO模型时不会报错
torch.serialization.add_safe_globals([doclayout_yolo.nn.tasks.YOLOv10DetectionModel])

# LangChain的DashScope嵌入
from langchain_community.embeddings.dashscope import DashScopeEmbeddings
# MinerU的文档解析工具
from mineru.cli.common import do_parse, read_fn
from pathlib import Path  # 路径处理
from dotenv import load_dotenv  # 环境变量加载

load_dotenv()  # 加载.env文件中的环境变量
```

2. 创建MinerURAGSystem类

实现对文件进行解析和存储，并创建检索方法，方便后面直接调用

2.1 初始化

```
class MinerURAGSystem:
    def __init__(self, persist_directory="./chroma_db"):
        # 初始化DashScope文本嵌入模型
        self.embedding_model = DashScopeEmbeddings(
            model=os.getenv("EMBEDDING_MODEL", "text-embedding-v2"),  # 从环境变量获取模型名
            dashscope_api_key=os.getenv("DASHSCOPE_API_KEY"),  # API密钥
        )
        # 创建ChromaDB持久化客户端
        self.client = chromadb.PersistentClient(path=persist_directory)
        # 获取或创建名为"documents"的集合
        self.collection = self.client.get_or_create_collection("documents")
```

2.2 文档处理（加载文档）

```
    def process_documents(self, pdf_directory, output_dir="./processed"):
        """
```

```
    将结构化内容分割成固定大小的块

    策略：
    1. 文本内容：按字数分割，考虑标题层级
    2. 表格内容：特殊处理，保持表格完整性
    3. 支持重叠(overlap)避免信息割裂
    """
        if not os.path.exists(output_dir):
            os.makedirs(output_dir)
        processed_files = []
        for filename in os.listdir(pdf_directory):  # 遍历 PDF 文件夹
            if filename.endswith('.pdf'):
                input_path = os.path.join(pdf_directory, filename)
                output_path = os.path.join(output_dir, filename.replace('.pdf', ''))
                pdf_file_name = Path(input_path).stem
                pdf_bytes = read_fn(input_path)  # 逐个读取 PDF二进制数据
                do_parse(  # 调用 MinerU 的 do_parse 进行智能解析
                    output_dir=output_path,
                    pdf_file_names=[pdf_file_name],
                    pdf_bytes_list=[pdf_bytes],
                    p_lang_list=["ch"],  # 默认使用中文
                    backend="pipeline"
                )
                content_file = os.path.join(output_path, pdf_file_name, "auto", f"
{pdf_file_name}_content_list.json")
                # 收集生成的 JSON 文件路径
                if os.path.exists(content_file):
                    processed_files.append(content_file)
        return processed_files  # 返回结果供后续处理
```

2.3 文本分块

```
    def _split_into_chunks(self, content_data, chunk_size, overlap):
        chunks = []
        current_chunk = []
        current_length = 0

        for item in content_data:
            if item.get('type') == 'text':
                text = item.get('text', '')
                if not text:
                    continue
                if item.get('text_level', 0) > 0:
                    if current_chunk:
                        chunks.append(
                            {'text': ' '.join(current_chunk), 'metadata': {'chunk_type':
'text', 'has_title': True}})
                    current_chunk = [f"## {text}"]
                    current_length = len(text)
                elif current_length + len(text) > chunk_size and current_chunk:
```

```
                    chunks.append({'text': ' '.join(current_chunk), 'metadata':
{'chunk_type': 'text'}})
                    current_chunk = [text]
                    current_length = len(text)
                else:
                    current_chunk.append(text)
                    current_length += len(text)

            elif item.get('type') == 'table':
                if current_chunk:
                    chunks.append({'text': ' '.join(current_chunk), 'metadata':
{'chunk_type': 'text'}})
                    current_chunk = []
                    current_length = 0

                table_body = item.get('table_body', '')
                if table_body:
                    if len(table_body) <= chunk_size:
                        chunks.append({
                            'text': table_body,
                            'metadata': {
                                'chunk_type': 'table',
                                'page_idx': item.get('page_idx', 0),
                                'has_caption': len(item.get('table_caption', [])) > 0,
                                'has_footnote': len(item.get('table_footnote', [])) > 0
                            }
                        })
                    else:
                        table_chunks = self._split_table_into_chunks(table_body,
chunk_size)
                        for i, chunk in enumerate(table_chunks):
                            chunks.append({
                                'text': chunk,
                                'metadata': {
                                    'chunk_type': 'table',
                                    'page_idx': item.get('page_idx', 0),
                                    'table_chunk_index': i,
                                    'total_table_chunks': len(table_chunks)
                                }
                            })
        if current_chunk:
            chunks.append({'text': ' '.join(current_chunk), 'metadata': {'chunk_type':
'text'}})
        return chunks
```

2.4 表格分块

```
    def _split_table_into_chunks(self, table_html, chunk_size):
        """
    专门处理HTML表格的分块
```

```
    特点:
    1. 保持表头在每个分块中
    2. 按行分割，避免拆分单个单元格
    3. 维护表格结构完整性
    """
        chunks = []
        if table_html.startswith('<table>'):
            table_content = table_html[7:-8]
        else:
            table_content = table_html
        rows = table_content.split('<tr>')
        table_head = rows[1]
        current_chunk = '<table>'+table_head
        print('表头:', current_chunk)

        current_chunk_size = len(current_chunk)
        for row in rows[2:]:
            row_content = '<tr>' + row
            if current_chunk_size + len(row_content) > chunk_size and current_chunk !=
'<table>':
                current_chunk += '</table>'
                chunks.append(current_chunk)
                current_chunk = '<table>' +table_head+ row_content
                current_chunk_size = len(current_chunk)
            else:
                current_chunk += row_content
                current_chunk_size += len(row_content)

        if current_chunk != '<table>'+table_head:
            current_chunk += '</table>'
            chunks.append(current_chunk)
        return chunks
```

2.5 向量化准备

```
def chunk_and_embed(self, content_files, chunk_size=512, overlap=50):
    """
    准备向量化所需的数据

    返回:
        documents: 文本内容列表
        metadatas: 元数据列表（包含来源、类型等信息）
        ids: 唯一标识符列表
    """
     documents = []
        metadatas = []
        ids = []
        for file_path in content_files:
            with open(file_path, 'r', encoding='utf-8') as f:
                content_data = json.load(f)
            doc_metadata = {
```

```
                'source': os.path.basename(file_path),
                'file_type': 'pdf',
                'processing_date': datetime.now().isoformat()
            }
            text_chunks = self._split_into_chunks(content_data, chunk_size, overlap)
            for i, chunk in enumerate(text_chunks):
                documents.append(chunk['text'])
                metadatas.append({**doc_metadata, **chunk['metadata']})
                ids.append(f"{os.path.basename(file_path)}_chunk_{i}")
        return documents, metadatas, ids
```

2.6 构建向量存储

```
def build_vector_store(self, documents, metadatas, ids):
    """
    将文档向量化并存储到ChromaDB

    特点:
    1. 先清空现有集合
    2. 批量添加新文档
    3. 使用DashScope生成文本嵌入
    """
    # 在删除之前，先检查集合中的文档数量是否大于0
        if self.collection.count() > 0:
            print(f"集合不为空，正在删除 {self.collection.count()} 个旧文档...")
            # 获取所有现有文档的ID
            existing_ids = self.collection.get(include=[])['ids']
            # 执行删除操作
            self.collection.delete(ids=existing_ids)
            print("旧文档删除完毕。")
        else:
            print("集合为空，无需删除。")

        # 现在，将新文档添加到干净的集合中
        embeddings = self.embedding_model.embed_documents(documents)
        self.collection.add(
            embeddings=embeddings,
            documents=documents,
            metadatas=metadatas,
            ids=ids
        )
        print(f"成功存储 {len(documents)} 个文档块到向量数据库")
```

2.7 文档检索

```
def query_documents(self, query_text, n_results=5):
    """
    检索与查询最相关的文档
```

```
    流程：
    1. 将查询文本向量化
    2. 在向量数据库中搜索相似文档
    3. 返回前n个最相关结果
    """
    query_embedding = self.embedding_model.embed_documents([query_text])[0]
        results = self.collection.query(
            query_embeddings=[query_embedding],
            n_results=n_results,
            include=['documents', 'metadatas', 'distances']
        )
        return results
```

3.构建RAGQASystem类实现文档进行智能问答。

3.1 初始化

```
class RAGQASystem:
    def __init__(self, mineru_rag_system):
        self.rag_system = mineru_rag_system  # 传入MinerURAGSystem实例
        dashscope.api_key = os.getenv("DASHSCOPE_API_KEY")  # 设置API密钥
```

3.2 构建提示词

```
def _build_prompt(self, question, context_docs):
    """
    构建给LLM的提示词

    提示词结构：
    1. 角色设定：财务报告分析师
    2. 任务说明：仅依据提供的上下文回答
    3. 上下文内容：检索到的文档片段
    4. 用户问题
    """
    context_text = "\n\n".join([f"文档片段 {i + 1}:\n{doc}" for i, doc in
enumerate(context_docs)])
        # 优化一下Prompt，让模型更好地理解任务
        prompt = f"""请你扮演一个专业的财务报告分析师。
                    根据下面提供的几段从《厦门灿坤实业股份有限公司2019年年度报告》中摘录的文字，
                    严谨且仅依据这些信息来回答用户的问题。
                    如果提供的信息不足以回答问题，请明确告知“根据现有信息无法回答”。

                    ---
                    相关文档内容：
                    {context_text}
                    ---

                    用户问题：{question}
                    """
        return prompt
```

3.3 调用LLM

```
def _call_llm(self, prompt):
    """
    调用DashScope的LLM API

    使用模型: qwen-turbo
    处理响应: 成功时提取回答，失败时返回错误信息
    """
    print("\n--- 正在向DashScope LLM发送请求... ---")

        # 使用通义千问模型
        response = dashscope.Generation.call(
            model='qwen-turbo',  # 或者 qwen-plus, qwen-max
            messages=[{'role': 'user', 'content': prompt}],
            result_format='message'  # 设置返回格式为 message
        )

        if response.status_code == HTTPStatus.OK:
            # 请求成功，提取模型的回答
            answer = response.output.choices[0]['message']['content']
            print("--- LLM响应成功 ---")
            return answer
        else:
            # 请求失败，打印错误信息
            print(f"请求失败: request_id={response.request_id}, status_code=
{response.status_code}, "
                  f"code={response.code}, message={response.message}")
            return "抱歉，调用大语言模型时出错，无法生成答案。"
```

3.4 生成答案

```
def generate_answer(self, question, context):
    """
    生成最终答案

    流程:
    1. 构建提示词
    2. 调用LLM
    3. 返回答案和上下文
    """
    prompt = self._build_prompt(question, context)
        # 打印将要发送的最终Prompt，方便调试
        print("\n--- 发送给LLM的最终Prompt ---\n", prompt)
        answer = self._call_llm(prompt)
        return answer, context
```

3.5 问答接口（对答案进行处理）

```
def ask_question(self, question):
    """
    完整的问答流程

    步骤:
    1. 检索相关文档
    2. 生成答案
    3. 返回答案和检索结果
    """
    results = self.rag_system.query_documents(question)
        if not results or not results.get('documents') or not results['documents'][0]:
            return "抱歉，没有找到相关的文档信息。", []

        # 将检索到的文档内容传递给 generate_answer
        answer, context = self.generate_answer(question, results['documents'][0])
        return answer, results
```

4.主程序流程

4.1 初始化系统

```
if __name__ == '__main__':

    # 1. 初始化RAG系统
    rag_system = MinerURAGSystem(persist_directory="./my_chroma_db")
```

4.2 文档处理逻辑

```
# 检查是否已存在解析结果
flag = True  # 是否需要解析PDF的标记
for file in os.listdir(folder_path):
    if file.endswith('.json'):
        # 如果已存在JSON文件，直接使用
        processed_files = glob.glob(os.path.join(folder_path, '**', '*_content_list.json'),
recursive=True)
        flag = False

if flag:
    # 需要解析PDF
    processed_files = rag_system.process_documents(pdf_directory=pdf_folder)
```

4.3 向量化流程（调用向量化函数进行向量）

```
# 文本分块与向量化准备
documents, metadatas, ids = rag_system.chunk_and_embed(content_files=processed_files)

# 构建向量数据库
rag_system.build_vector_store(documents, metadatas, ids)
```

4.4 问答系统

```
# 初始化问答系统
qa_system = RAGQASystem(mineru_rag_system=rag_system)

# 提问
question = "2019年一季度归属于上市公司股东的净利润? "
answer, search_results = qa_system.ask_question(question)
```